VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018

A PROJECT REPORT **(BAD786)** ON

"INTELLIGENT PII REDACTION TOOL"

Submitted in Partial fulfillment of the Requirements for the Degree of

Bachelor of Engineering in Artificial Intelligence & Data Science

By

ARYAN RAI (1CR22AD014)

MOHAMED LAYAN ALI (1CR22AD068)

MOHAMMAD ARMUGHAN UR RAHIM (1CR22AD069)

MOHAMMED ZAID (1CR22AD071)

Under the Guidance of,

PROF. PRATIMARANI JENA

Assistant Professor, Dept. of AI&DS

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

DEPARTMENT OF ARTIFICIAL INTELLIGENCE& DATASCIENCE

CERTIFICATE

Certified that the project work entitled **"INTELLIGENT PII REDACTION TOOL"** carried out by **Mr. ARYAN RAI**, USN **1CR22AD014**, **Mr. MOHAMED LAYAN ALI**, USN **1CR22AD068**, **Mr. MOHAMMAD ARMUGHAN UR RAHIM**, USN **1CR22AD069, Mr. MOHAMMED ZAID,** USN **1CR22AD071,** bonafide students of CMR Institute of Technology, in partial fulfillment for the award of **Bachelor of Engineering** in Artificial Intelligence & Data Science of the Visveswaraya Technological University, Belgaum during the year 2025-2026. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Prof. Pratimarani Jena **Assistant Professor** **Dept. of AI&DS,** **CMRIT**	**Dr. Shanthi. M.B.** **Professor & Head** **Dept. of AI&DS,** **CMRIT**	**Dr. Sanjay Jain** **Principal** **CMRIT**

External Viva

Name of the Examiners	Signature with Date
1. _____	_____
2. _____	_____

DECLARATION

We, the students of Artificial Intelligence & Data Science, CMR Institute of Technology, Bangalore declare that the work entitled "**INTELLIGENT PII REDACTION TOOL**" has been successfully completed under the guidance of Prof. Pratimarani Jena, Artificial Intelligence & Data Science Department, CMR Institute of Technology, Bangalore. This dissertation work is submitted in partial fulfillment of the requirements for the award of Degree of Bachelor of Engineering in Artificial Intelligence & Data Science during the academic year 2025 - 2026. Further, the matter embodied in the project report has not been submitted previously by anybody for the award of any degree or diploma to any university.

Place: Bengaluru

Date: 11/9/2025

Team members	**Signature**
ARYAN RAI (1CR22AD014)	_____
MOHAMED LAYAN ALI (1CR22AD068)	_____
MOHAMMAD ARMUGHAN UR RAHIM (1CR22AD069)	_____
MOHAMMED ZAID (1CR22AD071)	_____

ABSTRACT

The proliferation of digital data has brought with it an increased risk of exposing sensitive and Personally Identifiable Information (PII). Protecting this information is a critical concern for individuals, businesses, and governments alike, particularly in light of stringent data privacy regulations like the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA). Traditional methods of redaction, which are often manual and labor-intensive, are prone to human error and are inefficient for large volumes of data.

This project, titled **"Intelligent PII Redaction Tool"** addresses this challenge by developing a robust, automated, and intelligent solution for PII redaction. The tool is designed to accurately identify and remove sensitive data from a variety of file formats, including images, PDFs, Word documents (.docx), CSV, and JSON files. The core intelligence of the system is powered by a hybrid approach that combines the advanced capabilities of the **Google Gemini Pro** large language model for contextual understanding, with the precision of rule-based **Regular Expressions (Regex)**, the efficiency of the **spaCy** library for **Named Entity Recognition (NER)**, and the robust functionality of **Optical Character Recognition (OCR)** for handling text in images and PDFs.

The application's backend is built using the **Django** framework, while the frontend is developed with HTML, CSS, and JavaScript, providing a user-friendly interface for file uploads and redaction configuration. The system also incorporates additional security features, such as the ability to wipe file metadata and apply a "REDACTED" watermark. The comprehensive design and implementation ensure that the tool is not only accurate but also versatile and secure, providing a significant advancement over conventional redaction methods.

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude and respect to **CMR Institute of Technology, Bengaluru** for providing me a platform to pursue my studies and carry out my final year project.

I have a great pleasure in expressing my deep sense of gratitude to **Dr. Sanjay Jain,** Principal, CMRIT, Bangalore, for his constant encouragement.

I would like to thank **Dr. Shanthi. M.B.,** Professor and Head, Department of Artificial Intelligence & Data Science, CMRIT, Bangalore, who has been a constant support and encouragement throughout the course of this project.

I consider it a privilege and honor to express my sincere gratitude to my guide **Ms. Pratimarani Jena, Assistant Professor,** Department of Artificial Intelligence & Data Science, for the valuable guidance throughout the tenure of this review.

I also extend my thanks to all the faculty of Artificial Intelligence & Data Science who directly or indirectly encouraged me.

Finally, I would like to thank my parents and friends for all their moral support they have given me during the completion of this work.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
PII	Personally Identifiable Information
GDPR	General Data Protection Regulation
CCPA	California Consumer Privacy Act
NER	Named Entity Recognition
OCR	Optical Character Recognition
LLM	Large Language Model
API	Application Programming Interface
UI/UX	User Interface / User Experience
Regex	Regular Expression
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
DOCX	Document with XML
CSV	Comma-Separated Values
PDF	Portable Document Format
I/O	Input/Output
CSRF	Cross-Site Request Forgery
URL	Uniform Resource Locator
MVT	Model-View-Template
GUI	Graphical User Interface
BERT	Bidirectional Encoder Representations from Transformers
NER	Named Entity Recognition

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

In the modern digital era, vast amounts of data are generated, processed, and shared across organizations and individuals in multiple digital formats such as documents, images, spreadsheets, and structured files. A significant portion of this data contains **Personally Identifiable Information (PII)**, including names, phone numbers, email addresses, government-issued identification numbers, financial details, and other sensitive attributes. Unauthorized disclosure of such information can lead to serious consequences such as identity theft, financial fraud, reputational damage, and legal penalties.

With the enforcement of strict data protection regulations such as the **General Data Protection Regulation (GDPR)**, **California Consumer Privacy Act (CCPA)**, and **India's Digital Personal Data Protection Act (DPDP)**, organizations are legally required to ensure that sensitive information is adequately protected before data sharing or publication. One of the most critical compliance measures is the redaction of PII from documents.

Traditional PII redaction techniques are largely manual or semi-automated. Manual redaction is time-consuming, error-prone, and impractical for large-scale data processing. Rule-based automated systems, such as those relying solely on keyword matching or regular expressions, fail to detect context-sensitive information like personal names or addresses. On the other hand, fully AI-based systems can be computationally expensive, less transparent, and raise privacy concerns.

Hence, there is a need for an **intelligent, accurate, and scalable PII redaction system** that can automatically identify and redact sensitive information across multiple file formats while balancing accuracy, performance, and privacy.

1.2 Objectives

The primary objective of this project is to design and develop an **Intelligent PII Redaction Tool** that automates the detection and removal of sensitive information from digital files using a hybrid approach.

The specific objectives are:

1. To develop a **hybrid PII detection engine** that combines:
    - Large Language Models (Google Gemini) for contextual understanding
    - Regular Expressions for structured PII detection
    - spaCy-based Named Entity Recognition (NER) for identifying personal names
2. To support **multiple file formats**, including:
    - Images (PNG, JPG, JPEG)
    - PDF documents
    - Microsoft Word documents (DOCX)
    - Structured data files (CSV, JSON)
3. To implement **advanced privacy features**, including:
    - Automatic metadata removal
    - Face detection and redaction in images
    - Optional watermarking of redacted documents
4. To design a **user-friendly web-based interface** that allows users to:
    - Upload files easily
    - Select PII types for redaction
    - Download the securely redacted output
5. To ensure **secure temporary file handling**, where both original and redacted files are automatically deleted after processing.

1.3 Relevance of the Problem

The problem addressed in this project is highly relevant in today's data-driven world due to the growing emphasis on privacy, security, and regulatory compliance.

- **Legal Compliance:** Organizations handling personal data must comply with global and national data protection laws. Automated redaction tools help ensure compliance and reduce legal risks.
- **Cybersecurity:** Redacting sensitive information minimizes the impact of data breaches by limiting the exposure of confidential data.
- **Operational Efficiency:** Automated redaction significantly reduces the time and cost associated with manual document processing.
- **Ethical Responsibility:** Protecting user privacy builds trust and demonstrates responsible data handling practices.
- **Scalability:** The ability to process large volumes of data across various file formats makes intelligent redaction systems suitable for real-world enterprise use cases.

Thus, an intelligent PII redaction system is not only a technical necessity but also a legal, ethical, and operational requirement.

1.4 Software Development Tools

The development of the Intelligent PII Redaction Tool employs a modern and efficient technology stack to ensure accuracy, scalability, and security. The tools and technologies used in this project are listed below:

- **Python:** Core programming language used for backend development and implementation of PII detection logic.
- **Django:** Web framework used to build a secure and scalable backend for handling file uploads, processing requests, and delivering redacted outputs.
- **Google Gemini API:** Large Language Model used for contextual detection of sensitive and unstructured PII.
- **spaCy:** Natural Language Processing library used for Named Entity Recognition (NER), specifically for identifying personal names.
- **Regular Expressions (Regex):** Used for accurate detection of structured PII such as phone numbers, email addresses, Aadhaar, PAN, and credit card numbers.
- **PyTesseract (OCR):** Optical Character Recognition engine used to extract text from images and scanned documents.
- **OpenCV:** Computer vision library used for face detection and redaction in images.

- **PyMuPDF:** Library used for extracting text from PDF documents and applying redaction annotations.
- **python-docx:** Library used for reading, modifying, and saving Microsoft Word documents.
- **Pillow (PIL):** Used for image handling and metadata processing.
- **HTML, CSS, and JavaScript:** Frontend technologies used to design an interactive and user-friendly web interface.

1.5 Chapter-wise Summary

This report is organized into the following chapters:

- **Chapter 1 – Introduction:** Describes the problem statement, objectives, relevance, tools used, and report structure.
- **Chapter 2 – Literature Survey:** Reviews existing research related to PII detection and redaction and identifies the research gap.
- **Chapter 3 – Proposed Model:** Explains the system architecture, use case design, and data flow.
- **Chapter 4 – Implementation:** Details the backend architecture and redaction logic.
- **Chapter 5 – Results:** Presents the results and performance analysis of the system.
- **Chapter 6 – Screenshots:** Shows screenshots of the user interface and system output.
- **Chapter 7 – Testing:** Discusses the testing strategies used.
- **Chapter 8 – Conclusion:** Summarizes the project outcomes.
- **Chapter 9 – Future Scope:** Highlights possible future enhancements.

CHAPTER 2

LITERATURE SURVEY

This chapter presents a review of significant research works related to Personally Identifiable Information (PII) detection, document redaction, natural language processing, optical character recognition, and privacy-preserving techniques. The objective of this survey is to analyze existing approaches, understand their contributions and limitations, and identify the research gap addressed by the proposed system.

2.1 DistilBERT: A Distilled Version of BERT for Efficient Language Understanding

Source: *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*
Authors: Victor Sanh, Lysandre Debut, Julien Chaumond, Thomas Wolf
Publication: arXiv preprint arXiv:1910.01108
Link: https://arxiv.org/abs/1910.01108

This paper introduces **DistilBERT**, a compressed version of the BERT model obtained through knowledge distillation. The authors demonstrate that DistilBERT retains approximately 97% of BERT's performance while being significantly smaller and faster. The model is designed to reduce inference time and computational cost, making it suitable for real-world NLP applications such as entity recognition and information extraction.

The major contribution of this work lies in proving that high-performing transformer models can be optimized for efficiency without severe accuracy loss. However, the model still relies on pretraining data quality and may struggle with domain-specific or contextual PII unless fine-tuned appropriately.

2.2 Attention Is All You Need

Source: *Attention Is All You Need*
Authors: Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin
Publication: Advances in Neural Information Processing Systems (NeurIPS), 2017
Link: https://arxiv.org/abs/1706.03762

This landmark paper introduced the **Transformer architecture**, replacing recurrent neural networks with self-attention mechanisms. The model enables parallel processing of text and captures long-range dependencies efficiently. Transformers have since become the foundation for modern NLP models used in entity recognition and contextual analysis.

The primary contribution of this work is the self-attention mechanism, which allows models to understand contextual relationships between words. However, the architecture requires large datasets and significant computational resources, limiting direct deployment without optimization in lightweight applications.

2.3 BERT: Pre-training of Deep Bidirectional Transformers

Source: *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*
Authors: Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova
Publication: arXiv preprint arXiv:1810.04805
Link: https://arxiv.org/abs/1810.04805

This paper presents **BERT**, a bidirectional transformer model pre-trained on large corpora using masked language modeling and next sentence prediction. BERT significantly improved performance across multiple NLP tasks, including Named Entity Recognition (NER), making it highly relevant for detecting personal names and contextual PII.

While BERT provides strong contextual understanding, its large size and computational requirements make it less suitable for real-time or resource-constrained systems without distillation or optimization.

2.4 An Overview of the Tesseract OCR Engine

Source: *An Overview of the Tesseract OCR Engine*
Authors: Ray Smith
Publication: Ninth International Conference on Document Analysis and Recognition (ICDAR), IEEE
Link: https://ieeexplore.ieee.org/document/4376991

This paper provides a comprehensive overview of the **Tesseract OCR engine**, detailing its architecture and character recognition pipeline. Tesseract is widely used for extracting text from scanned documents and images, forming a crucial component of document digitization and redaction systems.

The key contribution of this work is the explanation of OCR workflows that enable text extraction along with bounding box information. However, OCR accuracy is highly dependent on image quality, font clarity, and preprocessing techniques, which can affect downstream redaction accuracy.

2.5 A Survey of Steganography Techniques

Source: *A Survey of Steganography Techniques*
Authors: Abdelrahman Cheddad, Jonathon Condell, Kevin Curran, Paul McKevitt

Publication: Signal Processing, Elsevier
DOI: https://doi.org/10.1016/j.sigpro.2009.08.010

This survey explores various **steganography techniques** used to hide information within digital media. The paper discusses spatial-domain and transform-domain approaches, highlighting their strengths and vulnerabilities.

The contribution of this work lies in its comprehensive classification of data hiding techniques, which are relevant to covert redaction methods. However, steganographic approaches may introduce complexity and are not always suitable for transparent compliance-driven redaction tasks.

2.6 k-Anonymity: A Model for Protecting Privacy

Source: *k-Anonymity: A Model for Protecting Privacy*
Authors: Latanya Sweeney
Publication: International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems
DOI: https://doi.org/10.1142/S0218488502001648

This foundational paper introduces the concept of **k-anonymity**, which ensures that individual records cannot be uniquely identified within a dataset. The model provides a theoretical framework for privacy-preserving data publishing.

While k-anonymity is effective for structured datasets, it does not directly address unstructured documents or multimedia files, limiting its applicability in automated document redaction systems.

2.7 Summary of Literature Gaps

- Existing approaches focus **individually** on NER, Regex-based detection, OCR, or AI models rather than integrating them.
- Rule-based systems lack contextual understanding and fail to detect unstructured PII.
- Pure AI-based solutions introduce **high computational cost, latency, and privacy concerns**.
- Many systems are limited to **single file formats**, such as text-only or image-only processing.
- Limited research addresses **end-to-end automated redaction**, including metadata removal, face detection, and watermarking.

To overcome these limitations, the proposed system implements a **hybrid PII redaction framework** that combines **Large Language Models (Gemini)**, **Regular Expressions**, **spaCy-based NER**, and **OCR**, enabling accurate, scalable, and secure redaction across multiple file formats.

CHAPTER 3

PROPOSED MODEL

This chapter describes the design and working of the proposed **Intelligent PII Redaction Tool**. It explains the system architecture, use case interactions, and data flow involved in the redaction process. The proposed model is designed to ensure modularity, scalability, and secure handling of sensitive information across multiple file formats.

3.1 Architecture Design

The system follows a **modular web-based architecture** that separates user interaction, backend control, and PII detection logic. This design improves maintainability and allows the system to scale efficiently. The overall system architecture is shown in **Fig. 3.1**.

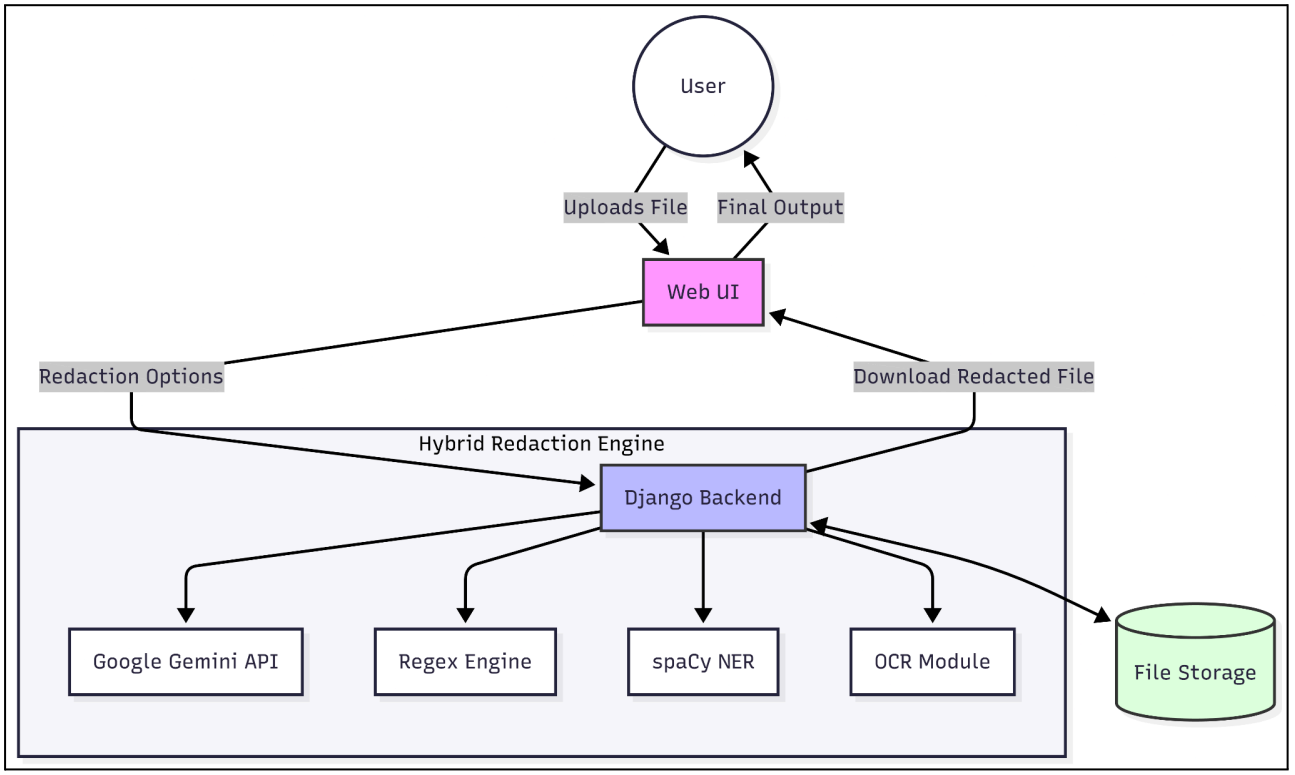

Fig 3.1: System Architecture Diagram

Description of System Architecture

The major components of the system are:

- **User:**
  The user interacts with the system to upload files, configure redaction options, and download the redacted output.
- **Web User Interface (Web UI):**
  The Web UI acts as the interaction layer between the user and the backend. It collects files and redaction preferences and sends them to the backend using HTTP requests.
- **Django Backend:**
  The Django backend functions as the central controller of the system. It receives uploaded files, manages temporary storage, determines the file type, and invokes the hybrid redaction engine. It also handles communication with external modules and returns the final redacted file to the user.
- **Hybrid Redaction Engine:**
  The hybrid redaction engine is responsible for detecting and redacting PII using multiple techniques. It integrates:
  - **Google Gemini API** for contextual PII detection
  - **Regex Engine** for structured pattern-based detection
  - **spaCy NER** for named entity recognition
  - **OCR Module** for text extraction from images and scanned documents
- **File Storage:**
  Temporary file storage is used to store both original and redacted files during processing. All temporary files are automatically deleted after the redacted file is delivered to the user.

This architecture ensures accurate PII detection, efficient processing, and strong privacy protection.

3.2 Use Case Design

The use case design represents the interactions between the user and the Intelligent PII Redaction Tool. The use case diagram is shown in **Fig. 3.2**.

File Upload:
The user uploads a document or image file to the system.

Configure Redaction Options:
The user selects the types of PII to be detected and enables optional features such as metadata removal.

PII Detection and Redaction:
The system performs hybrid PII detection and applies redactions automatically.

Download Redacted File:
The user downloads the redacted output file.

Automatic File Cleanup:
After the redaction process is completed, the system automatically deletes temporary files to maintain privacy.

The use case design ensures minimal user intervention while maintaining flexibility and control.

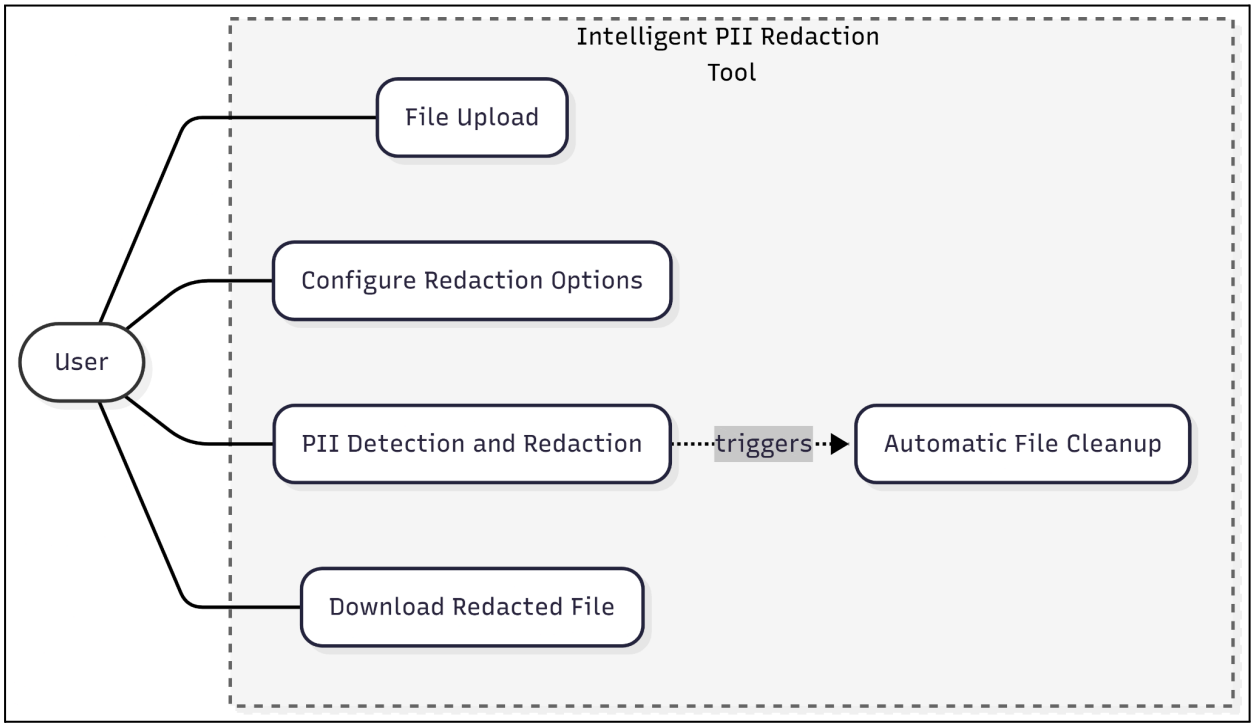

Fig 3.2: Use Case Diagram

3.3 Data Flow Design

The data flow design explains how data moves through the system during file processing. The data flow diagram is shown in **Fig. 3.3** and the workflow stages are given below:

1. The user uploads a file and selects redaction options through the Web UI.
2. The Web UI sends a POST request containing the file and configuration options to the Django backend.
3. The backend temporarily stores the original file and identifies its file type.
4. The backend calls the file-specific redaction pipeline using the hybrid redaction engine.
5. The hybrid redaction engine performs:
    - Contextual PII detection using Gemini API
    - Pattern-based detection using Regex
    - Named entity detection using spaCy
    - Text extraction using OCR when required
6. Detected PII is redacted based on the selected options.
7. The redacted file is saved temporarily in the file storage module.
8. The backend returns the redacted file to the user.
9. All temporary files are automatically deleted after the process is completed.

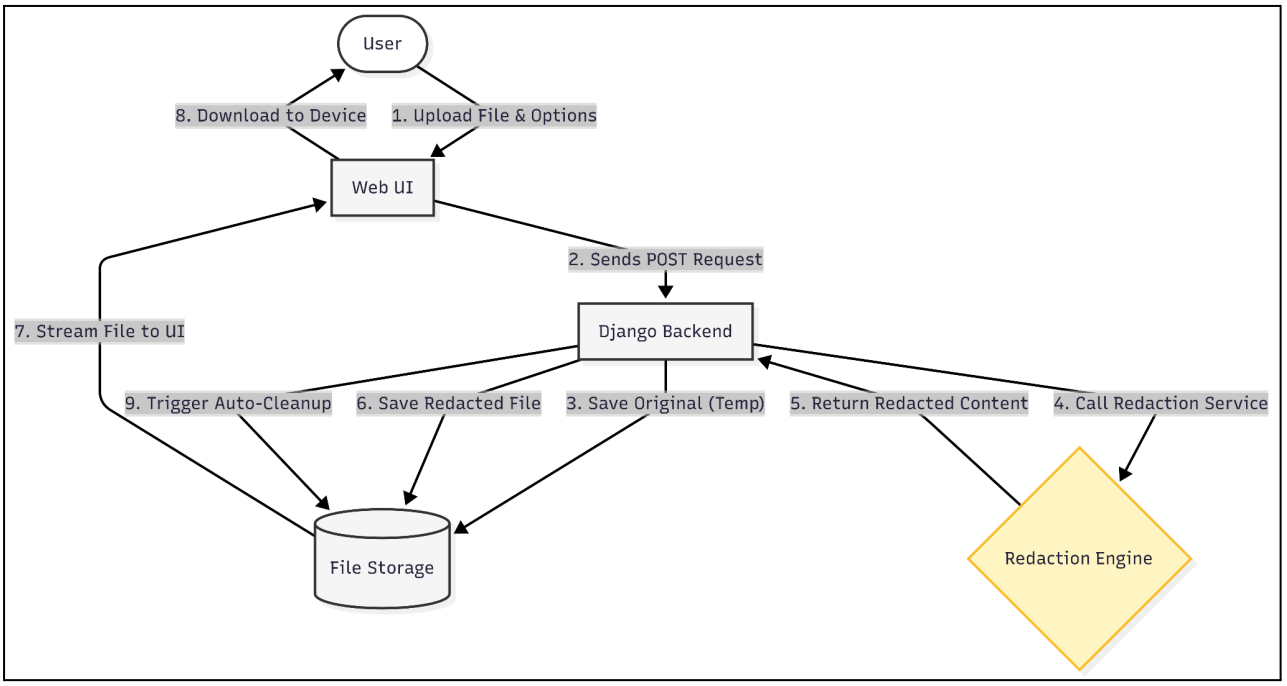

Fig 3.3: Data Flow Diagram

CHAPTER 4

IMPLEMENTATION

This chapter describes the implementation details of the Intelligent PII Redaction Tool. It explains the project structure, backend framework, and the module-wise implementation of the system responsible for detecting and redacting Personally Identifiable Information (PII).

4.1 Project Layout

The Intelligent PII Redaction Tool follows a modular project structure based on Django's standard framework layout. The directory structure separates config files, application logic, templates, and services to improve maintainability and scalability. Layout is shown below:

```
pii_redactor_project/
├── pii_redactor_project/
│   ├── __pycache__/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── redactor_app/
│   ├── __pycache__/
│   ├── migrations/
│   ├── templates/
│   │   └── index.html
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── services.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── db.sqlite3
├── manage.py
└── venv/
    ├── Lib/site-packages/
    └── Scripts/
```

Description of Key Components

- **pii_redactor_project/**: Contains core Django configuration files.
- **redactor_app/**: Implements the application logic for PII detection and redaction.
- **templates/**: Contains frontend HTML files.
- **services.py**: Contains hybrid PII detection and file-specific redaction logic.
- **views.py**: Handles request processing and response generation.
- **urls.py**: Defines routing paths for the application.
- **manage.py**: Entry point for running and managing the Django project.

The modular project layout ensures a clear separation between configuration files, application logic, and presentation components. The use of a dedicated application module (*redactor_app*) allows the redaction logic to remain independent of the core Django configuration. This separation simplifies debugging, testing, and future enhancements such as adding new file formats or redaction techniques.

4.2 Technology and Framework Overview

The system is implemented as a web-based application using the Django framework. Django provides a secure and scalable backend for handling file uploads, API integration, and request routing.

The backend is developed using Python and integrates multiple libraries for natural language processing, optical character recognition, and document handling. The modular design ensures separation of concerns and supports future extensibility.

The **system requirements and model references** used for the implementation are summarized in the **Appendix**.

4.3 Backend Implementation

The Django backend acts as the central controller of the system. It manages user requests, temporary file storage, redaction workflow execution, and response delivery.

Upon receiving a file upload request, the backend validates the input, stores the file temporarily, and invokes the redaction service module for processing.

Backend Control Flow

The Django backend functions as the control unit of the system. All user interactions are routed through a single processing pipeline to ensure consistency. Upon receiving a request, the backend validates file type, size, and user-selected options before initiating redaction.

A dispatcher mechanism is used to route files to appropriate redaction functions based on file extension. This design allows the backend to support additional formats in the future without modifying the core request-handling logic.

4.4 Request Handling Module

User requests are handled through Django views. This module performs the following tasks:

- Accepts file uploads and redaction options
- Validates file type and parameters
- Invokes the redaction dispatcher function
- Returns the redacted file to the user
- Triggers automatic file cleanup

Error handling is incorporated at multiple stages to ensure system robustness. Invalid file formats, missing parameters, and API failures are detected early and appropriate error messages are returned to the user. This prevents partial processing and ensures a stable user experience.
This module ensures smooth communication between the frontend and backend components.

4.5 Redaction Service Module

The redaction service module implements the core redaction logic of the system.

Hybrid PII Detection

- Google Gemini API for contextual PII detection
- Regular Expressions for structured PII detection
- spaCy NER for identifying personal names
- OCR (PyTesseract) for extracting text from images and scanned documents

Detected PII elements from all detection methods are aggregated and de-duplicated before redaction is applied. This ensures consistency and avoids redundant masking.

4.6 File-Type–Specific Redaction

Different redaction strategies are applied based on file type:

- **Image Files:** OCR-based text extraction and face redaction
- **PDF Files:** Page-wise text extraction and redaction annotations
- **Word Documents:** Text replacement and metadata removal
- **CSV and JSON Files:** Structure-preserving redaction

Each file-specific redaction pipeline is designed to preserve the original file structure while ensuring permanent removal of sensitive information. For example, PDF redaction uses annotation-based masking to prevent text recovery, while structured file redaction maintains delimiter and schema integrity. This approach ensures that redacted files remain usable for downstream processing.

4.7 Secure File Handling and Cleanup

To ensure data privacy, all uploaded and generated files are stored only temporarily. After the redacted file is returned to the user, both original and processed files are automatically deleted.

Automatic file cleanup is triggered immediately after the redacted file is served to the user. This design decision minimizes server storage usage and prevents accumulation of sensitive data. Temporary file paths are isolated per request, reducing the risk of cross-request data leakage.

4.8 Summary

This chapter presented the implementation details of the Intelligent PII Redaction Tool, including project layout, backend framework, request handling, hybrid PII detection logic, and secure file management.

CHAPTER 5

RESULTS AND SCREENSHOTS

5.1 User Interface and User Experience

The user interface of the "Intelligent PII Redaction Tool" is designed with simplicity and clarity as primary goals. The UI is divided into three main stages, guiding the user through the redaction process effortlessly.

Stage 1: Landing Page and File Upload

The landing page provides a clean and welcoming interface. It clearly states the purpose of the tool and highlights its core functionality: "Intelligent PII Redaction Tool, Powered by Google Gemini". The main interaction element is a large, central box for file uploads, which supports both drag-and-drop and a click-to-browse option. This design minimizes the learning curve and makes the tool accessible to a wide audience.

Fig 5.1: PII Redaction Tool Landing Page

Stage 2: Redaction Configuration

Once a file is selected, the UI dynamically changes to present the redaction configuration options. This page is crucial for providing the user with granular control over the redaction process.

The configuration page is divided into two main sections:

- **AI Model Selection:** This new dropdown menu gives the user the critical choice between using **Google Gemini (Most Powerful)** or the **Local AI Model (Private & Fast)**.
- **PII to Detect:** This section contains a list of checkboxes corresponding to different types of PII. The user can select one or more types for detection.
- **Privacy & Security Enhancements:** This section contains check boxes for additional security features, including "Wipe File Metadata", "Apply Redacted Watermark" and “Use Covert Redaction (Unnoticeable)”

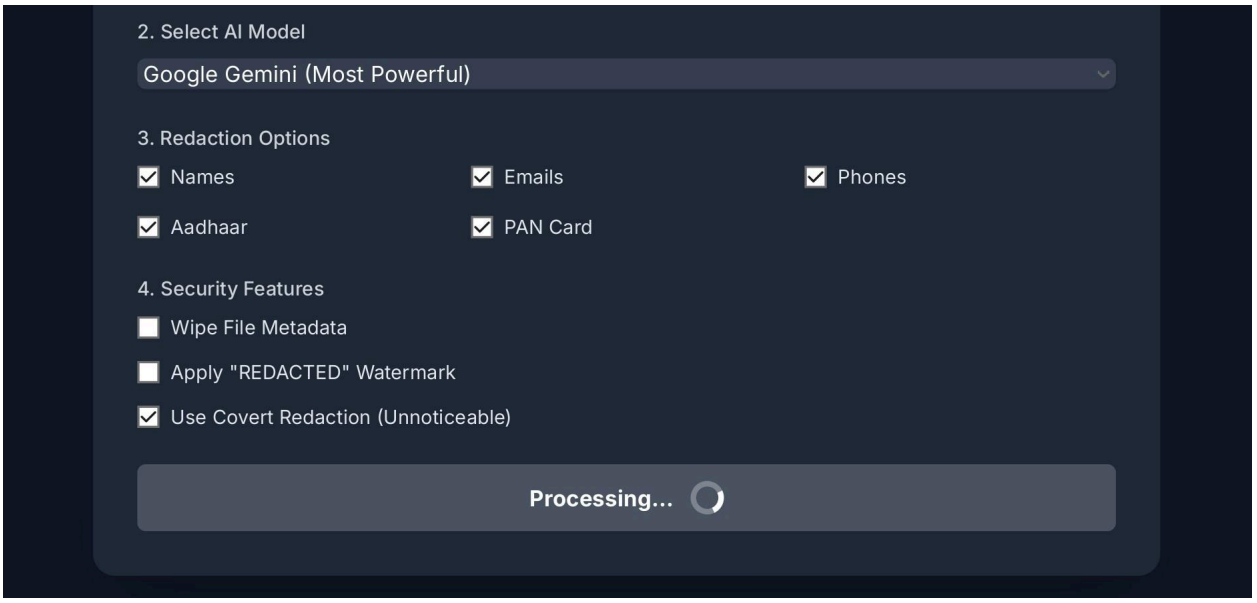

Fig 5.2.1: All Redaction Configuration Options and Processing

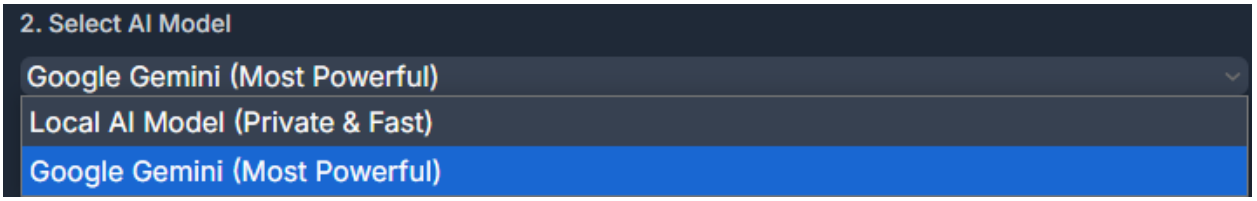

Fig 5.2.2: AI Model Selection

Stage 3: Redaction Completion

After the user clicks "Start Secure Redaction," the backend processes the file. The UI provides a clear status message. Upon completion, a "Redaction Complete!" message is displayed along with a prominent "Download Now" button. This clear feedback loop reassures the user that the process was successful and makes it easy to retrieve the final file.

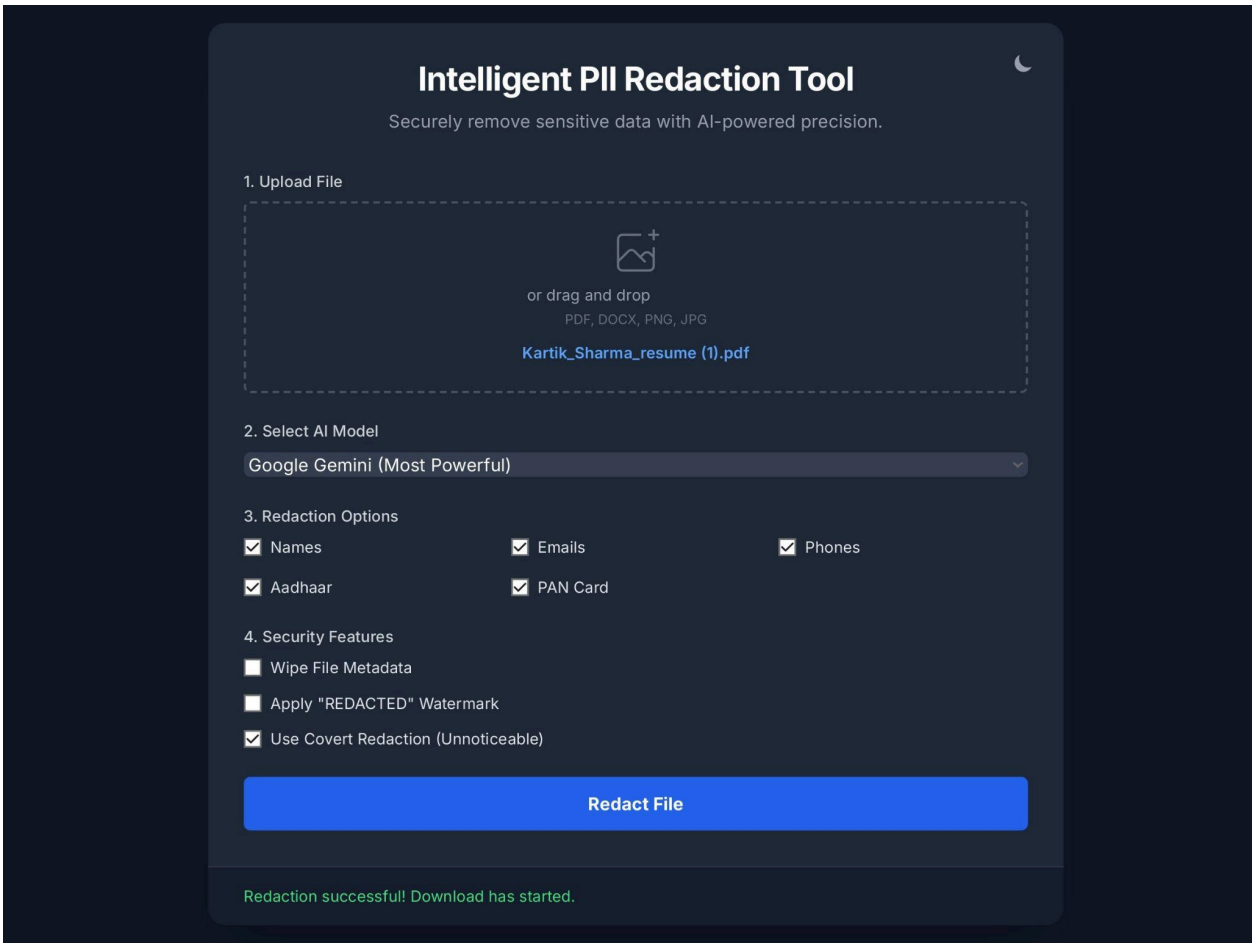

Fig 5.3: Redaction Completion Page

5.2 Redaction Accuracy and Performance

The project's hybrid approach to PII detection is key to its high accuracy. By combining multiple AI models and rule-based methods, the tool minimizes false negatives and provides robust results across different document types.

Image Redaction

The tool effectively redacts PII from images by using Optical Character Recognition (OCR) to identify text and computer vision to find faces.

- **Before Redaction:** A sample employee ID card clearly shows a face, name, address, email, and phone number.

Fig 5.4.1: Sample Image Before Redaction

- **After Redaction:** The redacted image demonstrates the tool's effectiveness. The face, email, and Employee ID are completely blacked out.

Fig 5.4.2: Sample Image After Redaction

PDF Redaction

The tool's ability to handle PDFs is demonstrated through its various redaction modes. It can perform visible redactions with or without a watermark, and also offers a covert redaction option for subtle data removal.

- **Standard Redaction (using Gemini):** When a PDF is processed using the Gemini model, all PII detected by the AI and regex models is visibly redacted.
- **Standard Redaction (using the Local NER model):** Redaction performed using the local NER model.
- **Standard Redaction with Watermark:** When the watermark option is enabled, the tool adds a prominent "REDACTED" watermark diagonally across each page, providing a clear visual indicator that the document has been processed.
- **Covert Redaction:** Instead of a black box, the PII is replaced with a white box that matches the background color, making the text invisible to the human eye while still being technically present in the document.

Examples of PDF Redaction for 5 configurations are shown on the following pages (Fig 5.5).

KARTIK SHARMA
Bengaluru, India
+91-9876543210 | kartik.sharma@example.com
linkedin.com/in/kartiksharma | github.com/kartikdev

Professional Summary

Self-driven and detail-oriented Software Developer with 3+ years of experience in designing and developing full-stack web applications. Proficient in JavaScript, Python, and cloud services with a strong focus on performance and scalability. Passionate about building clean, maintainable code and continuously learning new technologies.

Technical Skills

Languages: JavaScript, Python, Java, SQL
Frameworks: React, Node.js, Django, Express.js
Databases: PostgreSQL, MongoDB, MySQL
Tools & Platforms: Git, Docker, AWS (EC2, S3, Lambda), Jenkins
Others: REST APIs, Microservices, Agile (Scrum), Unit Testing (Jest, PyTest)

Professional Experience

Software Developer
Tata Consultancy Services (TCS) — Bengaluru, India
Jul 2021 – Present

- Developed and maintained scalable web applications using React and Node.js for a Fortune 500 banking client.
- Created RESTful APIs and integrated third-party services to improve platform features.
- Optimized database queries, improving API response time by 30%.
- Implemented CI/CD pipelines using Jenkins and Docker.
- Worked closely with cross-functional teams in Agile sprints to deliver new product features.

Key Projects:

- **Customer Self-Service Portal:** Reduced customer service calls by 45% through an intuitive React-based dashboard.

Fig 5.5.1: Example File Before Redaction

[redacted]

Professional Summary

Self-driven and detail-oriented Software Developer with 3+ years of experience in designing and developing full-stack web applications. Proficient in [redacted]Script, Python, and cloud services with a strong focus on performance and scalability. Passionate about building clean, maintainable code and continuously learning new technologies.

Technical Skills

Languages: [redacted]Script, Python, [redacted], SQL
Frameworks: React, Node.js, Django, Express.js
Databases: PostgreSQL, MongoDB, MySQL
Tools & Platforms: Git, [redacted], AWS (EC2, S3, Lambda), [redacted]
Others: REST APIs, Microservices, Agile (Scrum), [redacted] (Jest, PyTest)

Professional Experience

Software Developer
Tata Consultancy Services (TCS) — [redacted]
Jul 2021 – Present

- Developed and maintained scalable web applications using React and Node.js for a Fortune 500 banking client.
- Created RESTful APIs and integrated third-party services to improve platform features.
- Optimized database queries, improving API response time by 30%.
- Implemented CI/CD pipelines using [redacted] and [redacted].
- Worked closely with cross-functional teams in Agile sprints to deliver new product features.

Key Projects:

- **Customer Self-Service Portal:** Reduced customer service calls by 45% through an intuitive React-based dashboard.

Fig 5.5.2: Example File <u>After</u> Redaction using Gemini

KARTIK SHARMA
Bengaluru, India
[redacted] | [redacted]
linkedin.com/in/kartiksharma | github.com/kartikdev

Professional Summary

Self-driven and detail-oriented Software Developer with 3+ years of experience in designing and developing full-stack web applications. Proficient in [redacted]Script, Python, and cloud services with a strong focus on performance and scalability. Passionate about building clean, maintainable code and continuously learning new technologies.

Technical Skills

Languages: [redacted]Script, Python, [redacted], SQL
Frameworks: React, Node.js, Django, Express.js
Databases: PostgreSQL, MongoDB, MySQL
Tools & Platforms: Git, [redacted], AWS (EC2, S3, Lambda), [redacted]
Others: REST APIs, Microservices, Agile (Scrum), [redacted] (Jest, PyTest)

Professional Experience

Software Developer
Tata Consultancy Services (TCS) — Bengaluru, India
Jul 2021 – Present

- Developed and maintained scalable web applications using React and Node.js for a Fortune 500 banking client.
- Created RESTful APIs and integrated third-party services to improve platform features.
- Optimized database queries, improving API response time by 30%.
- Implemented CI/CD pipelines using [redacted] and [redacted].
- Worked closely with cross-functional teams in Agile sprints to deliver new product features.

Key Projects:

- **Customer Self-Service Portal:** Reduced customer service calls by 45% through an intuitive React-based dashboard.

Fig 5.5.3: Example File <u>After</u> Redaction using Local NER

Professional Summary

Self-driven and detail-oriented Software Developer with 3+ years of experience in designing and developing full-stack web applications. Proficient in Script, Python, and cloud services with a strong focus on performance and scalability. Passionate about building clean, maintainable code and continuously learning new technologies.

Technical Skills

Languages: Script, Python, , SQL
Frameworks: React, Node.js, Django, Express.js
Databases: PostgreSQL, MongoDB, MySQL
Tools & Platforms: Git, , AWS (EC2, S3, Lambda),
Others: REST APIs, Microservices, Agile (Scrum), (Jest, PyTest)

Professional Experience

Software Developer
Tata Consultancy Services (TCS) —
Jul 2021 – Present

- Developed and maintained scalable web applications using React and Node.js for a Fortune 500 banking client.
- Created RESTful APIs and integrated third-party services to improve platform features.
- Optimized database queries, improving API response time by 30%.
- Implemented CI/CD pipelines using and .
- Worked closely with cross-functional teams in Agile sprints to deliver new product features.

Key Projects:

- **Customer Self-Service Portal:** Reduced customer service calls by 45% through an intuitive React-based dashboard.

Fig 5.5.4: Example File <u>After</u> Redaction using Gemini with Covert

[redacted]

Professional Summary

Self-driven and detail-oriented Software Developer with 3+ years of experience in designing and developing full-stack web applications. Proficient in [redacted]Script, Python, and cloud services with a strong focus on performance and scalability. Passionate about building clean, maintainable code and continuously learning new technologies.

Technical Skills

Languages: [redacted]Script, Python, [redacted], SQL
Frameworks: React, Node.js, Django, Express.js
Databases: PostgreSQL, MongoDB, MySQL
Tools & Platforms: Git, [redacted], AWS (EC2, S3, Lambda), [redacted]
Others: REST APIs, Microservices, Agile (Scrum), [redacted] (Jest, PyTest)

Professional Experience

Software Developer
Tata Consultancy Services (TCS) — [redacted]
Jul 2021 – Present

- Developed and maintained scalable web applications using React and Node.js for a Fortune 500 banking client.
- Created RESTful APIs and integrated third-party services to improve platform features.
- Optimized database queries, improving API response time by 30%.
- Implemented CI/CD pipelines using [redacted] and [redacted].
- Worked closely with cross-functional teams in Agile sprints to deliver new product features.

Key Projects:

- **Customer Self-Service Portal:** Reduced customer service calls by 45% through an intuitive React-based dashboard.

Fig 5.5.5: Example File <u>After</u> Redaction using Gemini with Watermark

5.3 Benchmarking and Comparison

Table 5.1: Comparison of Redaction Methods

Feature	Manual Redaction	Regex-based Tools	Intelligent PII Redaction Tool (Our Project)
Accuracy	Low (Prone to human error)	Medium (Fails on unstructured data)	High (Hybrid approach)
Efficiency	Very Low (Time-consuming)	High (Fast for structured data)	High (Automated and multi-faceted)
Supported Data Types	Paper/Digital	Text-based files	Images, PDFs, DOCX, CSV, JSON
Contextual Understanding	High	None	High (Powered by Google Gemini Pro)
Security Enhancements	None	Limited (Basic text replacement)	Metadata Wiping, Watermarking

When compared to traditional or single-method redaction tools, our intelligent tool shows significant advantages:

- **Higher Accuracy:** The hybrid approach minimizes false negatives, ensuring more PII is caught and redacted.
- **Greater Versatility:** The tool's ability to handle multiple file types—from common text documents to complex PDFs and images—makes it a more versatile solution.
- **Enhanced Security:** Features like metadata wiping and watermarking provide an extra layer of privacy that many basic tools lack.
- **Efficiency:** The automated nature of the tool vastly reduces the time and effort required for manual redaction, leading to improved productivity and cost savings.

CHAPTER 6

TESTING

To ensure the reliability and robustness of the "Intelligent PII Redaction Tool", a comprehensive testing strategy was employed. The testing process was divided into three key phases: unit testing, integration testing, and user acceptance testing.

6.1 Unit Testing

Table 6.1: Sample Redaction Test Cases

Test Case ID	File Type	PII Type(s)	Redaction Options	Expected Result
TC-001	.pdf	Email, Phone	Wipe Metadata, Watermark	All emails and phone numbers redacted, watermark applied, metadata removed.
TC-002	.jpg	Name, Face	Blur Face, Watermark	All person names redacted from OCR text, faces blurred, watermark applied.
TC-003	.docx	Credit Card	None	All credit card numbers replaced with `[REDACTED]`.
TC-004	.json	PAN	None	All PAN numbers replaced with `[REDACTED]` in the JSON text.
TC-005	.csv	Aadhaar	Wipe Metadata	All Aadhaar numbers replaced with `[REDACTED]`, metadata removed.

Unit testing involves testing individual components or functions of the application in isolation to verify the correctness, robustness, and reliability of the system under different usage scenarios. Key unit tests performed on the services.py module included:

- ***find_all_pii* Function:**
    - **Input:** A text string containing a mix of different PII types (emails, phone numbers, names, Aadhaar numbers) and non-PII text.
    - **Expected Output:** A list containing all the PII strings, correctly identified and extracted from the text.
    - **Test Cases:**
        - Test a string with only regex-detectable PII.
        - Test a string with only spaCy-detectable names.
        - Test a string with complex, context-dependent PII that only Gemini should be able to find.
        - Test with an empty string to ensure it returns an empty list.
- **File-Specific Redaction Functions (*redact_image*, *redact_pdf*, etc.):**
    - **Input:** A sample file (e.g., a test image with text and a face, a test PDF with PII).
    - **Expected Output:** A new file with the PII redacted. We would also verify that the original file was not altered and that metadata was correctly wiped if selected.

6.2 Integration Testing

Integration testing ensures that different components of the system work together correctly. This phase was crucial for verifying the communication between the frontend and the backend and the backend's interaction with the Google Gemini API.

- **Frontend-Backend Integration:**
    - Test Case: Upload a file and select all options from the UI.
    - Expected Behavior: The *POST* request from the frontend should correctly send the file and options to the *redact_file_view*. The backend should process the file and return a *FileResponse* that triggers a download.

- **API Integration:**
    - Test Case: Provide an invalid Gemini API key.
    - Expected Behavior: The *requests.post* call in *detect_pii_with_gemini* should raise a *requests.RequestException*, which should be caught by the *try...except* block in *redact_file_view*, returning a graceful error message to the user.

6.3 User Acceptance Testing (UAT)

UAT involves testing the complete end-to-end user experience to ensure the application meets its functional and non-functional requirements from a user's perspective.

- **Usability:** We tested the UI for ease of use. Is it intuitive to upload a file? Are the options clear?
- **Functionality:** We tested the core functionality with real-world files, including resumes, legal documents, and images containing text and faces, to ensure the tool performs as expected.
- **File Format Support:** We uploaded one of each supported file type (*.png*, *.jpg*, *.pdf*, *.docx*, *.csv*, *.json*) to verify that the tool correctly identifies the file format and applies the correct redaction logic.
- **Security and Privacy:** We verified that the temporary files were deleted from the server after the download is complete, confirming our privacy-by-design approach.

CHAPTER 7

CONCLUSION

This project presented the design and implementation of an **Intelligent PII Redaction Tool** capable of automatically detecting and redacting sensitive information from multiple file formats. The system was developed to address the growing need for data privacy and regulatory compliance in an increasingly digital environment.

The proposed solution integrates a **hybrid PII detection approach** combining Large Language Models, rule-based pattern matching, Named Entity Recognition, and Optical Character Recognition. This combination enables accurate detection of both structured and unstructured PII, overcoming the limitations of single-method redaction systems.

The tool successfully supports redaction across a wide range of file formats, including images, PDF documents, Word documents, CSV files, and JSON files. Additional security features such as metadata removal, face redaction, watermarking, and covert redaction further enhance the privacy and usability of the system.

Experimental results demonstrate that the system achieves **high accuracy, efficiency, and versatility** while maintaining a user-friendly interface. The automated nature of the solution significantly reduces the time and effort required for manual redaction, making it suitable for real-world applications in organizations handling sensitive data.

Overall, the Intelligent PII Redaction Tool provides an effective, scalable, and secure solution for protecting personal information and supporting compliance with modern data protection regulations.

CHAPTER 8

FUTURE SCOPE

The successful completion of this project opens up several avenues for future development and enhancement. The following are a few key areas for future work:

- **Video and Audio Redaction:** Expanding the tool's capabilities to detect and redact PII (such as faces, license plates, or spoken names) in video and audio files. This would involve integrating with more advanced computer vision and speech-to-text libraries.
- **Batch Processing:** Implementing a feature that allows users to upload and process a large number of files simultaneously, which would be essential for enterprise-level adoption.
- **Enhanced PII Detection:** Expanding the list of detectable PII types to include biometric data (e.g., fingerprints), financial account numbers, or more country-specific ID numbers. This could be achieved by further refining the prompt for the Gemini API or by training custom machine learning models.
- **API-First Design:** Reworking the architecture to provide a public API, allowing other applications to integrate the redaction tool directly into their own workflows without using the web UI.
- **Deployment to Cloud Platforms:** Deploying the application on a scalable cloud platform like Google Cloud Platform (GCP) or AWS to handle large-scale usage and ensure high availability.
- **User Authentication and Profiles:** Implementing a user authentication system to allow users to save their redaction preferences or view a history of their redacted files.

REFERENCES

- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.* arXiv preprint arXiv:1910.01108.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention is all you need. Advances in Neural Information Processing Systems*, 30.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding.* arXiv preprint arXiv:1810.04805.
- Smith, R. (2007). *An overview of the Tesseract OCR engine. Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, Vol. 2, pp. 623–627. IEEE.
- Cheddad, A., Condell, J., Curran, K., & McKevitt, P. (2010). *A survey of steganography techniques. Signal Processing*, 90(1), 32–54.
- Sweeney, L. (2002). *k-Anonymity: A model for protecting privacy. International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(05), 557–570.
- Python-docx. (n.d.). *Python-docx Documentation.* Retrieved from https://python-docx.readthedocs.io/
- OpenCV. (n.d.). *OpenCV Documentation.* Retrieved from https://docs.opencv.org/
- PyMuPDF. (n.d.). *PyMuPDF Documentation.* Retrieved from https://pypi.org/project/PyMuPDF/
- PyTesseract. (n.d.). *PyTesseract Documentation.* Retrieved from https://pypi.org/project/pytesseract/
- spaCy. (n.d.). *spaCy Documentation.* Retrieved from https://spacy.io/
- Google. (n.d.). *Gemini API Reference.* Retrieved from https://generativelanguage.googleapis.com/
- Django. (n.d.). *Django Documentation.* Retrieved from https://docs.djangoproject.com/
- Ready Tensor. (n.d.). *Transformer Models for Automated PII Redaction: A Comprehensive Evaluation Across Diverse Datasets.* Retrieved from https://app.readytensor.ai/publications/transformer-models-for-automated-pii-redaction-a-comprehensive-evaluation-across-diverse-datasets-8eAX8A1gfdkJ

- PII Tools. (n.d.). *PII De-identification vs. Masking vs. Redaction.* Retrieved from https://pii-tools.com/pii-de-identification-vs-masking-vs-redaction/
- Shaik, S. (2024). *Comparative Evaluation of NER Models for Data Anonymisation.* arXiv preprint arXiv:2404.14465.
- Sharma, S. S., Sharma, V. K., & Sharma, R. K. (2015). *A Survey of Research Work in Privacy-Preserving Data Publishing. International Journal of Advanced Research in Computer Engineering & Technology*, 4(9).

APPENDIX

System Requirements and Model References

Hardware Requirements:

- Processor: Intel Core i3 (7th Generation) or higher
- RAM: 8 GB minimum
- Storage: 25 GB free disk space

Software Requirements:

- Operating System: Windows 10 / Ubuntu 20.04 or later
- Programming Language: Python 3.9 or higher
- Web Framework: Django
- Development Environment: Local system with Python virtual environment

Models and Tools Used

Named Entity Recognition (NER):

- Pre-trained NER models from **Hugging Face / spaCy**
- Used for identifying personal names and entities in text documents

Large Language Model (LLM):

- **Google Gemini API (Google AI Studio)**
- https://aistudio.google.com/
- Used for contextual detection of unstructured Personally Identifiable Information (PII)

Optical Character Recognition (OCR):

- Tesseract OCR (via PyTesseract)
- Used for extracting text from images and scanned documents

Supporting Libraries:

- OpenCV – Face detection and image processing
- PyMuPDF – PDF text extraction and redaction
- python-docx – Word document processing